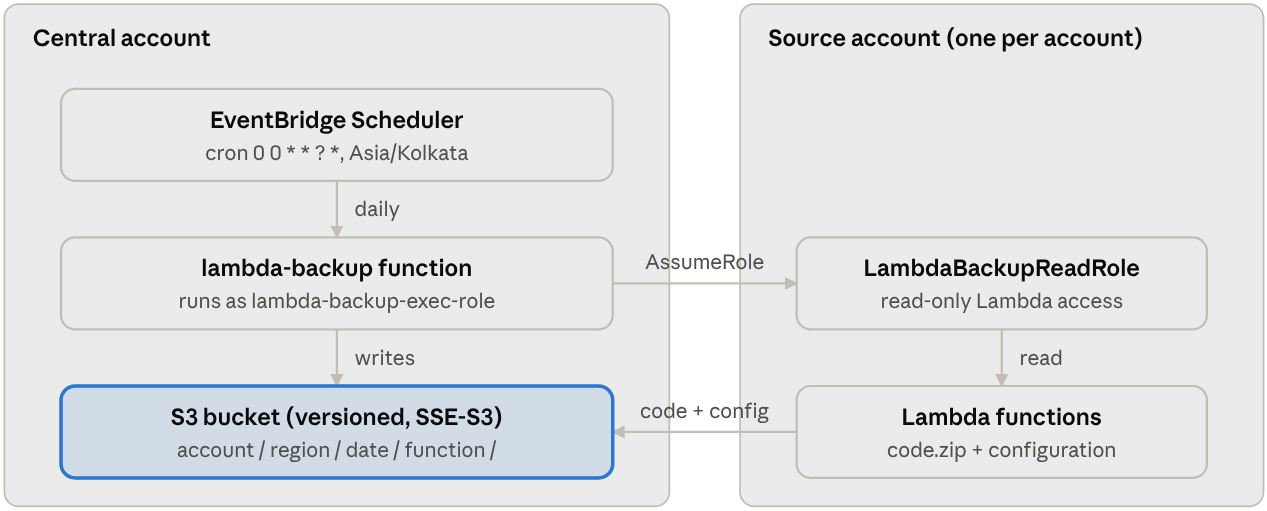
Lambda Cross-Account Backup Runbook

Oct 24, 2026 · @jan

Overview

A scheduled Lambda in the central account assumes a read-only role in each source account, exports every function's code and configuration, and writes them to a central S3 bucket. AWS Backup does not support Lambda as a resource type, so this export is the backup. Everything is done in the AWS Console; the only code is the backup function you paste in Phase 4.

One central function pulls every account's Lambda into one bucket



architecture · central account pulls from each source account

The source accounts only grant read access; all writing happens inside the central account, so no cross-account bucket policy is needed.

Phase	Account	What you build	Output you carry forward
1	Central	S3 backup bucket	Bucket name
2	Central	Execution role for the backup function	Role ARN
3	Each source	`LambdaBackupReadRole`	Role ARN per source account

Phase	Account	What you build	Output you carry forward
4	Central	`lambda-backup` function	Deployed function with env vars
5	Central	Test run	Backup files in S3
6	Central	Daily schedule	Schedule ARN, first automatic run
7	Both (optional)	ECR replication, restore steps	Container images in central ECR

Do the phases in order: Phase 3's trust policy points at the role created in Phase 2, so that role must exist first.

Phase 1: Create the backup bucket (central account)

This phase gives you a private, versioned, encrypted bucket that will hold every backup.

1. Open **S3 → Create bucket**.
2. Bucket name: `lambda-backups-<central-account-id>`. Region: the one you work in (e.g. `ap-south-1`).
3. Leave **Block all public access** on.
4. **Bucket Versioning**: Enable.
5. **Default encryption**: SSE-S3 (Amazon S3 managed keys), the default.
6. Click **Create bucket**.
7. Optional: open the bucket → **Management** tab → **Create lifecycle rule** (e.g. transition to Glacier after 30 days, expire after 365 days).
8. Recommended: **Permissions** tab → **Bucket policy → Edit**, paste the policy below (replace the bucket name) → Save. It refuses any access that isn't over HTTPS.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyInsecureTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::lambda-backups-<central-account-id>",
      "arn:aws:s3:::lambda-backups-<central-account-id>/*"
    ],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  }]
}
```

Keep `s3:GetObject` on this bucket limited to a small backup/admin group. If you need backups nobody can delete, enable **Object Lock** when creating the bucket; it can't be added later.

Expected output

- The bucket appears in the S3 list with Access shown as "Bucket and objects not public".
- **Properties** tab shows Versioning: Enabled and Default encryption: SSE-S3.
- Note down: **bucket name** (used in Phases 2 and 4).

- [x] ~~Phase 1 done~~

Phase 2: Create the execution role (central account)

This role is what the backup function runs as; it can assume the source roles and write to the bucket.

1. Open **IAM → Roles → Create role**.
2. Trusted entity: **AWS service → Lambda** → Next.
3. Attach `AWSLambdaBasicExecutionRole` → Next.
4. Role name: `lambda-backup-exec-role` → **Create role**.
5. Open the role → **Add permissions → Create inline policy → JSON**, paste the policy below (replace the bucket name), name it `lambda-backup-access`, save.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": [
        "arn:aws:iam::111111111111:role/LambdaBackupReadRole",
        "arn:aws:iam::222222222222:role/LambdaBackupReadRole"
      ]
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::<your-bucket-name>/*"
    }
  ]
}
```

List one ARN per source account; never use `*` for the account. If you already finished this phase with the wildcard version, edit the inline policy and replace it with exact ARNs.

Expected output

- Role summary shows two policies: `AWSLambdaBasicExecutionRole` (AWS managed) and `lambda-backup-access` (inline).
- **Trust relationships** tab shows `lambda.amazonaws.com` as the principal.
- Note down: **role ARN**, e.g. `arn:aws:iam::<CENTRAL_ACCOUNT_ID>:role/lambda-backup-exec-role` (used in Phase 3).

- [x] ~~Phase 2 done~~

Phase 3: Create the read-only role (each source account)

Repeat this phase in every account whose Lambda functions you want to back up. The role name must be exactly `LambdaBackupReadRole`, because Phase 2's policy and the code both expect it.

1. Sign in to the source account → **IAM → Roles → Create role**.
2. Trusted entity: **Custom trust policy**. Paste:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::<CENTRAL_ACCOUNT_ID>:role/lambda-backup-exec-role" },
    "Action": "sts:AssumeRole"
  }]
}
```

3. Next → skip managed policies → Next.
4. Role name: `LambdaBackupReadRole` → **Create role**.
5. Open the role → **Add permissions → Create inline policy → JSON**, paste the policy below, name it `lambda-backup-read`, save.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "lambda:ListFunctions",
      "lambda:GetFunction",
      "lambda:GetFunctionConfiguration",
      "lambda:GetPolicy",
      "lambda:ListAliases",
      "lambda:ListVersionsByFunction",
      "lambda:ListEventSourceMappings",
      "lambda:GetLayerVersion",
      "ec2:DescribeRegions"
    ],
    "Resource": "*"
  }]
}
```

Expected output

- **Trust relationships** tab shows the central account's `lambda-backup-exec-role` ARN as the trusted principal.
- **Permissions** tab shows the inline policy `lambda-backup-read`.
- Note down: **source account ID** for each account (used in Phase 4's `SOURCE_ACCOUNTS`).

- [ ] Phase 3 done in every source account

Phase 4: Create the backup function (central account)

This phase deploys the function that does the export. It discovers every enabled region in each source account automatically, so no region list is needed. It also redacts environment variable values by default, verifies each package's size and SHA-256 before storing it, backs up layers and published versions, and writes a summary of every run to the bucket.

4a. Create the function

1. Open **Lambda → Create function → Author from scratch**.
2. Name: `lambda-backup`. Runtime: **Python 3.12**.
3. Expand **Change default execution role** → **Use an existing role** → `lambda-backup-exec-role`.
4. Click **Create function**.

4b. Paste the code

In the **Code** tab, replace the contents of `lambda_function.py` with the code below, then click **Deploy**.

```
"""
lambda-backup
Exports Lambda functions (configuration, code, layers, aliases, triggers,
versions, resource policy) from source accounts into a central S3 bucket.

Runs in the CENTRAL account. Assumes LambdaBackupReadRole in each source
account and discovers every enabled region automatically.

Environment variables
  BACKUP_BUCKET     required  central bucket name
  SOURCE_ACCOUNTS   required  comma-separated 12-digit account IDs
  SOURCE_ROLE       optional  role name in source accounts (default
LambdaBackupReadRole)
  REDACT_ENV        optional  "true" (default) masks environment variable
values
                              "false" stores them (only if bucket access is
locked down)
  BACKUP_LAYERS     optional  "true" (default) also stores layer zip files

S3 layout
  <account>/<region>/<run-id>/<function>/function.json
  <account>/<region>/<run-id>/<function>/extras.json
```

```
  <account>/<region>/<run-id>/<function>/code.zip
  <account>/<region>/<run-id>/_layers/<layer-name>-v<version>.zip
  _runs/<run-id>.json                       (run summary / audit record)
"""

import base64
import datetime
import hashlib
import json
import logging
import os
import re
import tempfile
import urllib.error
import urllib.request
from urllib.parse import urlparse

import boto3
from botocore.config import Config

logger = logging.getLogger()
logger.setLevel(logging.INFO)


# ---------------------------------------------------------------------------
# Configuration (validated once, at cold start)
# ---------------------------------------------------------------------------

def _flag(name, default):
    return os.environ.get(name, default).strip().lower() == "true"


BUCKET = os.environ["BACKUP_BUCKET"].strip()
ROLE = os.environ.get("SOURCE_ROLE", "LambdaBackupReadRole").strip()
REDACT_ENV = _flag("REDACT_ENV", "true")
BACKUP_LAYERS = _flag("BACKUP_LAYERS", "true")
HOME_REGION = os.environ.get("AWS_REGION", "us-east-1")

if not re.fullmatch(r"[a-z0-9][a-z0-9.-]{1,61}[a-z0-9]", BUCKET):
    raise ValueError("BACKUP_BUCKET is not a valid S3 bucket name")
if not re.fullmatch(r"[\w+=,.@-]{1,64}", ROLE):
    raise ValueError("SOURCE_ROLE is not a valid IAM role name")
```

```
ACCOUNTS = sorted({a.strip() for a in
os.environ["SOURCE_ACCOUNTS"].split(",") if a.strip()})
if not ACCOUNTS:
    raise ValueError("SOURCE_ACCOUNTS is empty")
for _a in ACCOUNTS:
    if not re.fullmatch(r"\d{12}", _a):
        raise ValueError(f"Invalid account ID in SOURCE_ACCOUNTS: {_a!r}")

BOTO_CFG = Config(
    retries={"max_attempts": 10, "mode": "adaptive"},  # handles API
throttling
    connect_timeout=10,
    read_timeout=60,
)
DOWNLOAD_TIMEOUT_S = 120
CHUNK = 1024 * 1024
TIME_RESERVE_MS = 90_000          # stop starting new work with 90 s left
ALLOWED_HOST_SUFFIX = ".amazonaws.com"
SAFE_NAME = re.compile(r"[A-Za-z0-9_.-]{1,140}")

s3 = boto3.client("s3", config=BOTO_CFG)
sts = boto3.client("sts", region_name=HOME_REGION, config=BOTO_CFG)


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------

class OutOfTime(Exception):
    """Raised when the invocation is close to its timeout."""


class _NoRedirect(urllib.request.HTTPRedirectHandler):
    """Refuse redirects so a download can never be steered to another
host."""

    def redirect_request(self, req, fp, code, msg, headers, newurl):
        raise urllib.error.HTTPError(req.full_url, code, "Redirects are not
allowed", headers, fp)


_OPENER = urllib.request.build_opener(_NoRedirect)
```

```
def _session_for(account):
    creds = sts.assume_role(
        RoleArn=f"arn:aws:iam::{account}:role/{ROLE}",
        RoleSessionName="lambda-backup",
    )["Credentials"]
    return boto3.Session(
        aws_access_key_id=creds["AccessKeyId"],
        aws_secret_access_key=creds["SecretAccessKey"],
        aws_session_token=creds["SessionToken"],
    )


def _enabled_regions(sess):
    ec2 = sess.client("ec2", region_name=HOME_REGION, config=BOTO_CFG)
    return sorted(r["RegionName"] for r in ec2.describe_regions()["Regions"])


def _paginate(client, op, key, **kwargs):
    items = []
    for page in client.get_paginator(op).paginate(**kwargs):
        items.extend(page.get(key, []))
    return items


def _safe(name):
    if not SAFE_NAME.fullmatch(name):
        raise ValueError(f"Unexpected characters in name: {name!r}")
    return name


def _put_json(key, data, owner):
    s3.put_object(
        Bucket=BUCKET,
        Key=key,
        Body=json.dumps(data, default=str, indent=2).encode("utf-8"),
        ContentType="application/json",
        ExpectedBucketOwner=owner,          # never write into someone else's
bucket
        ChecksumAlgorithm="SHA256",         # S3 verifies integrity on
arrival
    )
```

```
def _download_verified(url, expected_sha256_b64, expected_size):
    """Download to /tmp over HTTPS from AWS only, then verify size and SHA-
256."""
    parsed = urlparse(url)
    host = (parsed.hostname or "").lower()
    if parsed.scheme != "https" or not host.endswith(ALLOWED_HOST_SUFFIX):
        raise ValueError(f"Refusing download from unexpected host {host!r}")

    digest = hashlib.sha256()
    size = 0
    fd, path = tempfile.mkstemp(dir="/tmp", suffix=".zip")
    try:
        with os.fdopen(fd, "wb") as out, _OPENER.open(url,
timeout=DOWNLOAD_TIMEOUT_S) as resp:
            for chunk in iter(lambda: resp.read(CHUNK), b""):
                digest.update(chunk)
                size += len(chunk)
                out.write(chunk)

        if expected_size is not None and size != expected_size:
            raise ValueError(f"Size mismatch: got {size}, expected
{expected_size}")
        if expected_sha256_b64:
            actual = base64.b64encode(digest.digest()).decode("ascii")
            if actual != expected_sha256_b64:
                raise ValueError("SHA-256 mismatch: downloaded package is not
the deployed one")
        return path
    except Exception:
        os.remove(path)
        raise


def _upload_zip(path, key, owner):
    try:
        s3.upload_file(
            path,
            BUCKET,
            key,
            ExtraArgs={
                "ContentType": "application/zip",
                "ExpectedBucketOwner": owner,
```

```
                "ChecksumAlgorithm": "SHA256",
            },
        )
    finally:
        os.remove(path)


def _redact(info):
    env = info.get("Configuration", {}).get("Environment", {})
    if REDACT_ENV and env.get("Variables"):
        env["Variables"] = {k: "***REDACTED***" for k in env["Variables"]}


# ---------------------------------------------------------------------------
# Backup of one function / layer
# ---------------------------------------------------------------------------

def _backup_layers(lam, info, base, owner, seen_layers, warnings):
    for layer in info.get("Configuration", {}).get("Layers", []):
        arn = layer["Arn"]
        if arn in seen_layers:
            continue
        seen_layers.add(arn)
        parts = arn.split(":")              #
arn:aws:lambda:region:acct:layer:name:version
        try:
            name, version = _safe(parts[6]), _safe(parts[7])
            content = lam.get_layer_version_by_arn(Arn=arn)["Content"]
            path = _download_verified(content["Location"],
content.get("CodeSha256"), content.get("CodeSize"))
            _upload_zip(path, f"{base}/_layers/{name}-v{version}.zip", owner)
        except Exception as e:  # third-party layers are often not readable
            warnings.append(f"layer {arn}: {type(e).__name__}: {e}")
            logger.warning("Layer not backed up %s: %s", arn, e)


def _backup_function(lam, base, name, owner, seen_layers, warnings):
    prefix = f"{base}/{_safe(name)}"

    info = lam.get_function(FunctionName=name)
    info.pop("ResponseMetadata", None)
    code = info.get("Code", {})
    code_url = code.pop("Location", None)   # pre-signed URL: never stored
```

```
    _redact(info)

    extras = {
        "aliases": _paginate(lam, "list_aliases", "Aliases",
FunctionName=name),
        "versions": _paginate(lam, "list_versions_by_function", "Versions",
FunctionName=name),
        "event_sources": _paginate(lam, "list_event_source_mappings",
"EventSourceMappings", FunctionName=name),
    }
    for v in extras["versions"]:            # version configs also carry env
vars
        if REDACT_ENV and v.get("Environment", {}).get("Variables"):
            v["Environment"]["Variables"] = {k: "***REDACTED***" for k in
v["Environment"]["Variables"]}
    try:
        extras["resource_policy"] =
json.loads(lam.get_policy(FunctionName=name)["Policy"])
    except lam.exceptions.ResourceNotFoundException:
        extras["resource_policy"] = None

    package = info.get("Configuration", {}).get("PackageType", "Zip")
    if package == "Zip" and code.get("RepositoryType") == "S3" and code_url:
        cfg = info["Configuration"]
        path = _download_verified(code_url, cfg.get("CodeSha256"),
cfg.get("CodeSize"))
        _upload_zip(path, f"{prefix}/code.zip", owner)
    elif package == "Image":
        warnings.append(f"{prefix}: container image ({code.get('ImageUri')}),
use ECR replication")

    _put_json(f"{prefix}/function.json", info, owner)
    _put_json(f"{prefix}/extras.json", extras, owner)

    if BACKUP_LAYERS:
        _backup_layers(lam, info, base, owner, seen_layers, warnings)


# ---------------------------------------------------------------------------
# Handler
# ---------------------------------------------------------------------------

def lambda_handler(event, context):
```

```
    owner = sts.get_caller_identity()["Account"]
    run_id = datetime.datetime.now(datetime.timezone.utc).strftime("%Y-%m-
%dT%H%M%SZ")
    count, failures, warnings = 0, [], []
    incomplete = False

    def check_time():
        if context.get_remaining_time_in_millis() < TIME_RESERVE_MS:
            raise OutOfTime()

    try:
        for acct in ACCOUNTS:
            check_time()
            try:
                sess = _session_for(acct)
                regions = _enabled_regions(sess)
            except Exception as e:
                failures.append(f"{acct}: {type(e).__name__}: {e}")
                logger.error("Account %s skipped: %s", acct, e)
                continue

            for region in regions:
                check_time()
                base = f"{acct}/{region}/{run_id}"
                seen_layers = set()
                try:
                    lam = sess.client("lambda", region_name=region,
config=BOTO_CFG)
                    functions = _paginate(lam, "list_functions", "Functions")
                except Exception as e:
                    failures.append(f"{acct}/{region}: {type(e).__name__}:
{e}")
                    logger.error("Region %s/%s skipped: %s", acct, region, e)
                    continue

                for fn in functions:
                    check_time()
                    name = fn["FunctionName"]
                    try:
                        _backup_function(lam, base, name, owner, seen_layers,
warnings)
                        count += 1
                        logger.info("Backed up %s/%s/%s", acct, region, name)
```

```
                    except Exception as e:
                        failures.append(f"{acct}/{region}/{name}:
{type(e).__name__}: {e}")
                        logger.error("Failed %s/%s/%s: %s", acct, region,
name, e)
    except OutOfTime:
        incomplete = True
        logger.error("Stopped early: invocation close to timeout")

    summary = {
        "run_id": run_id,
        "accounts": ACCOUNTS,
        "functions_backed_up": count,
        "failures": failures,
        "warnings": warnings,
        "incomplete": incomplete,
        "env_vars_redacted": REDACT_ENV,
    }
    try:
        _put_json(f"_runs/{run_id}.json", summary, owner)
    except Exception as e:
        logger.error("Could not write run summary: %s", e)

    logger.info(json.dumps({k: v for k, v in summary.items() if k not in
("failures", "warnings")}))

    if failures or incomplete:
        # Fail the invocation so the Lambda "Errors" metric and alarm fire.
        raise RuntimeError(
            f"Backup run {run_id}: {len(failures)} failures, incomplete=
{incomplete}. "
            f"See s3://{BUCKET}/_runs/{run_id}.json"
        )
    return {"run_id": run_id, "functions_backed_up": count, "warnings":
len(warnings)}
```

4c. Configure it

1. **Configuration → General configuration → Edit**: Timeout `15 min 0 sec`, Memory `512 MB`, Ephemeral storage `1024 MB` → Save. The extra storage is where each zip is verified before upload.

2. **Configuration → Concurrency → Edit**: Reserve concurrency `1` → Save. Two runs can then never overlap.
3. **Configuration → Environment variables → Edit**, add the required keys (optional ones only if you need to change the default):

Key	Required	Example / default	Notes
`BACKUP_BUCKET`	Yes	`lambda-backups-<central-account-id>`	Must be a bucket in this account
`SOURCE_ACCOUNTS`	Yes	`111111111111,222222222222`	12-digit IDs only
`REDACT_ENV`	No	`true`	`false` stores secret values; only with tight bucket access
`BACKUP_LAYERS`	No	`true`	`false` skips layer zips
`SOURCE_ROLE`	No	`LambdaBackupReadRole`	Change only if you named the role differently

Expected output

- Green banner: "Successfully updated the function lambda-backup".
- **Configuration → General** shows Timeout 15 min, Memory 512 MB, Ephemeral storage 1024 MB; Concurrency shows Reserved 1; Handler is `lambda_function.lambda_handler`.
- **Environment variables** lists at least the two required keys above.

☐ Phase 4 done

Phase 5: Test run and validation (central account)

Run the function once by hand and confirm the files land in S3 before you schedule it.

1. In `lambda-backup`, open the **Test** tab.
2. Create a new event named `manual-run` with body `{}` → **Test**.
3. Open the bucket in S3 and browse the folders.

Expected output

Execution result shows **Succeeded** with a response like:

```
{
  "run_id": "2026-10-24T183000Z",
  "functions_backed_up": 12,
  "warnings": 0
}
```

The log output lists one `Backed up <account>/<region>/<function>` line per function. In S3 you should see this layout:

```
lambda-backups-<central-account-id>/
├── _runs/
|   └── 2026-10-24T183000Z.json        (run summary: counts, failures,
warnings)
└── 111111111111/
    └── ap-south-1/
        └── 2026-10-24T183000Z/            (run ID, UTC timestamp)
            ├── _layers/
            |   └── my-layer-v3.zip
            └── my-function/
                ├── function.json   (runtime, handler, VPC, layers, tags; env
values redacted)
                ├── extras.json     (aliases, versions, triggers, resource
policy)
                └── code.zip        (deployment package, SHA-256 verified)
```

The run ID is in UTC, so a midnight-IST run shows the previous day's date with time `1830`. Open `_runs/<run-id>.json` first: it lists every failure and warning for that run.

Container-image functions have no `code.zip`; Phase 7 covers them.

If it fails

Error in result or logs	Likely cause	Fix
`RuntimeError: Backup run ... N failures`	Some accounts, regions or functions failed	Open `_runs/<run-id>.json` for the list, fix each cause below
`incomplete=True` in the error	Run neared the 15-minute timeout	One schedule per account, each with its own `SOURCE_ACCOUNTS`

Error in result or logs	Likely cause	Fix
`Invalid account ID in SOURCE_ACCOUNTS` (init error)	Typo or non-12-digit ID	Fix the env var
`AccessDenied` on `AssumeRole`	Role name or ARN mismatch between Phase 2 and Phase 3	Check the trust policy ARN, the role name, and that the account is listed in the Phase 2 policy
`UnauthorizedOperation` on `DescribeRegions`	Source role lacks region discovery	Add `ec2:DescribeRegions` to the Phase 3 policy
`AccessDenied` on `PutObject`	Bucket name wrong, or bucket not owned by this account	Match `BACKUP_BUCKET` and the Phase 2 policy resource
`SHA-256 mismatch` or `Size mismatch`	Package changed or corrupted during download	Re-run; if it repeats, check the function in the source account
`Refusing download from unexpected host`	Download URL not on amazonaws.com	Treat as suspicious and investigate
Layer warning in summary	Layer owned by a third party or not readable	Expected for public layers; note the ARN for restore
`functions_backed_up: 0`	Wrong account ID or no functions in any enabled region	Check `SOURCE_ACCOUNTS` and the Lambda console in each account

- [ ] Phase 5 done: files visible in S3

Phase 6: Schedule the daily backup (central account)

EventBridge Scheduler runs the function every night at midnight IST.

1. Open **Amazon EventBridge → Scheduler → Schedules → Create schedule**.
2. Name: `lambda-backup-daily`.
3. Schedule pattern: **Recurring schedule → Cron-based schedule**, `0 0 * * ? *`.
4. Time zone: **Asia/Kolkata**. Flexible time window: **Off** → Next.
5. Target: **AWS Lambda Invoke** → function `lambda-backup` → payload `{}` → Next.

6. Permissions: **Create new role for this schedule** → Next → **Create schedule**.

Add a failure alarm

1. Open **CloudWatch → Alarms → Create alarm → Select metric → Lambda → By Function Name →** `lambda-backup` **→ Errors**.
2. Statistic **Sum**, period **1 day**, condition **Greater/Equal 1**.
3. Notification: create or choose an SNS topic with your email → confirm the subscription email.
4. Name: `lambda-backup-failed` → **Create alarm**.

Expected output

- Schedule `lambda-backup-daily` shows Status **Enabled**.
- Schedule details show the next 10 trigger dates, each at 00:00 Asia/Kolkata.
- After the first night: a new run folder (e.g. `2026-10-24T183000Z/`) appears under each account and region, a matching summary file appears in `_runs/`, and `lambda-backup` → **Monitor** shows one invocation with no errors.
- Alarm `lambda-backup-failed` shows state **OK** once it has data; you get an email if any run reports a failure or stops early.

☐ Phase 6 done: first scheduled run confirmed

Phase 7 (optional): Container images and restore

7a. Container-image functions

These return only an ECR image URI, so replicate the images to the central account.

1. **Central account → ECR → Private registry → Permissions → Generate statement**: choose cross-account replication and add each source account ID → Save.
2. **Each source account → ECR → Private registry → Replication → Edit → Add rule**: destination type **Cross-account**, destination account = central account ID, region = your region → Save.

Expected output: new image pushes in the source account appear as repositories in the central account's ECR within a few minutes. Replication applies to images pushed after the rule is created; push existing images again to copy them.

7b. Restoring a function

1. In S3, download `code.zip`, `function.json` and `extras.json` for the function and date you need.
2. In the target account, open **Lambda → Create function → Author from scratch**, using the runtime from `Configuration.Runtime` and an appropriate execution role.
3. **Code → Upload from → .zip file** → upload `code.zip`.
4. From `function.json`, set Handler, Environment variables, Timeout, Memory, VPC and Layers in the **Configuration** tabs.
5. From `extras.json`, re-create aliases, triggers (event source mappings) and resource-based permissions.

With `REDACT_ENV=true` (the default), environment variable values show as `***REDACTED***`: re-enter them from your secrets store or the team that owns the function. Layers used by the function are in the same run's `_layers/` folder; publish them first under **Lambda → Layers → Create layer**, then attach them. To restore an older published version, use its entry in `extras.json` → `versions`; only the current code (`$LATEST`) is stored as `code.zip`.

Expected output: the restored function's **Test** run returns the same result as the original.